NAME :- Lokare Mahir Abdul Majeed

ROLL NO :- 23AI34

BATCH :- 02

EXPERIMENT NO :- 01

AIM :-

Implement Multilayer Perceptron (MLP) to simulate XOR gate and explore TensorFlow/PyTorch frameworks.

OBJECTIVE :-

1. To understand the concept of Multilayer Perceptron (MLP) in Deep Learning.
2. To understand the architecture and working of a neural network with a hidden layer.
3. To implement an MLP for simulating the XOR logic gate.
4. To understand the role of activation functions in solving non-linear problems.
5. To implement the XOR problem using TensorFlow and PyTorch.
6. To compare the basic implementation and predictions obtained using both frameworks.

THEORY :-

Multilayer Perceptron (MLP) :-

A Multilayer Perceptron (MLP) is a type of artificial neural network consisting of an input layer, one or more hidden layers, and an output layer. Each neuron receives inputs, applies weights and biases, and passes the result through an activation function.

A basic MLP can be represented as: Input Layer → Hidden Layer → Output Layer

The hidden layer allows the network to learn non-linear relationships in the input data. Non-linear activation functions are important because a network containing only linear operations behaves like a single linear model regardless of the number of layers.

XOR Gate :-

The XOR (Exclusive OR) gate produces an output of 1 when the two inputs are different and produces an output of 0 when the two inputs are the same.

Input X1	Input X2	XOR Output
0	0	0
0	1	1
1	0	1
1	1	0

The XOR problem is a non-linear classification problem. A single-layer perceptron cannot correctly separate the XOR outputs using a single linear decision boundary. Therefore, a hidden layer with a non-linear activation function is required.

MLP Architecture for XOR :-

The MLP used in this experiment consists of: Input Layer (2 neurons) → Hidden Layer (4 neurons) → Output Layer (1 neuron)

The two input neurons represent X1 and X2. The hidden layer learns the non-linear relationship between the inputs, while the output neuron produces the final XOR prediction.

Mathematical Formulation :-

For the hidden layer: $Z1 = X \cdot W1 + b1$, $A1 = \sigma(Z1)$

For the output layer: $Z2 = A1 \cdot W2 + b2$, $\hat{Y} = \sigma(Z2)$

where: X represents the input, W represents weights, b represents biases, σ represents the sigmoid activation function, $\hat{Y}$ represents the predicted output.

The sigmoid activation function is: $\sigma(x) = 1 / (1 + e^{-x})$. It produces values between 0 and 1, making it suitable for binary classification.

Loss Function :-

Binary Cross Entropy (BCE) can be used to measure the difference between the actual XOR output and the predicted output.

$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$

During training, the network adjusts its weights and biases to reduce the loss. The forward pass produces predictions, while the backward pass calculates gradients and updates the parameters.

TensorFlow and PyTorch :-

TensorFlow and PyTorch are deep learning frameworks that provide tools for creating, training, and evaluating neural networks. In this experiment, both frameworks are used to implement the same MLP architecture and solve the XOR classification problem.

PROCEDURE :-

1. Install the required Python libraries.
2. Import TensorFlow and PyTorch.
3. Define the XOR input and output dataset.
4. Create an MLP model with two input neurons, one hidden layer, and one output neuron.
5. Apply a non-linear activation function in the hidden layer.
6. Use sigmoid activation in the output layer.
7. Define the loss function and optimizer.
8. Train the MLP using TensorFlow.
9. Test the trained model using the XOR inputs.
10. Implement the same MLP architecture using PyTorch.
11. Train the PyTorch model using the XOR dataset.
12. Test the PyTorch model and display the predictions.
13. Compare the predicted outputs with the expected XOR outputs.

CODE :-

```
import numpy as np
import tensorflow as tf
import torch
import torch.nn as nn
import torch.optim as optim

np.random.seed(42)
tf.random.set_seed(42)
torch.manual_seed(42)

X = np.array([
    [0, 0],
```

```
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=np.float32)

y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=np.float32)

tf_model = tf.keras.Sequential([
    tf.keras.layers.Dense(4, activation="sigmoid", input_shape=(2,)),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

tf_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

tf_model.fit(X, y, epochs=2000, verbose=0)
tf_predictions = tf_model.predict(X, verbose=0)
tf_classes = (tf_predictions >= 0.5).astype(int)

print("TensorFlow Predictions:")
for i in range(len(X)):
    print(X[i].astype(int), "->", round(float(tf_predictions[i][0]), 4), "->",
tf_classes[i][0])

class XORNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(2, 4),
            nn.Sigmoid(),
            nn.Linear(4, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.network(x)

X_torch = torch.tensor(X)
y_torch = torch.tensor(y)

torch_model = XORNet()
criterion = nn.BCELoss()
optimizer = optim.Adam(torch_model.parameters(), lr=0.1)

for epoch in range(2000):
    predictions = torch_model(X_torch)
    loss = criterion(predictions, y_torch)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

with torch.no_grad():
    torch_predictions = torch_model(X_torch)
    torch_classes = (torch_predictions >= 0.5).int()

print("\nPyTorch Predictions:")
for i in range(len(X)):
```

```
    print(X[i].astype(int), "->", round(float(torch_predictions[i][0]), 4), "->",
torch_classes[i][0].item())
```

OUTPUT :-

```
TensorFlow Predictions:
[0 0] -> 0.0001 -> 0
[0 1] -> 0.9999 -> 1
[1 0] -> 0.9999 -> 1
[1 1] -> 0.0001 -> 0

PyTorch Predictions:
[0 0] -> 0.0001 -> 0
[0 1] -> 0.9996 -> 1
[1 0] -> 1.0 -> 1
[1 1] -> 0.0004 -> 0
```

CONCLUSION :-

The experiment successfully implemented an MLP to simulate the XOR gate using a hidden layer and non-linear activation function. The model was implemented using TensorFlow and PyTorch, and both successfully produced the expected XOR outputs.